JavaScript para Web-Apps

JavaScript como motor de interactividad, modelo de ejecución, estructura de aplicación, eventos y DOM

1. **modelo de ejecución** de JavaScript en el navegador (call stack, event loop, tasks/microtasks) con mentalidad de ingeniería.
2. interactividad básica: **eventos**, handlers, estado local y actualización de UI.
3. Manipular el **DOM** de forma segura y eficiente
4. Organizar código para una Web-App: **módulos**, separación de responsabilidades, estructura de ficheros (evitar "app.js monolítico").

Qué aporta JS en el stack del curso

- HTML: estructura
- CSS: presentación
- **JS: comportamiento y lógica**
  - interacción (click, input, gestures)
  - estado (filtros, favoritos)
  - render dinámico (a partir de datos)
  - comunicación con APIs (próximas sesiones)

JS en el navegador: runtime simplificado

Componentes:

- Motor JS (ejecuta código)
- Web APIs (DOM, timers, storage, fetch…)
- Event loop (coordina callbacks)
- UI thread (render/layout)

Es **clave para entender cómo funciona realmente JavaScript en una Web-App**

JavaScript **no controla directamente el navegador**.
No dibuja píxeles.
No accede a red por sí solo.
No crea ventanas.
Lo que hace es orquestar APIs que el navegador le ofrece.
Es un sistema coordinado.

Componentes del runtime

Motor JavaScript

Ejemplos reales:

- V8 (Chrome)
- SpiderMonkey (Firefox)

- JavaScriptCore (Safari)

¿Qué hace?

- Interpreta y ejecuta tu código JS
- Gestiona memoria (garbage collection)
- Maneja la pila de ejecución (call stack)

Importante

El motor JS **solo ejecuta código JavaScript puro**.
No sabe qué es el DOM.
No sabe qué es un setTimeout.
No sabe qué es fetch.
Eso pertenece a las Web APIs.

Web APIs

Son funcionalidades que el navegador expone a JavaScript.
Ejemplos importantes en nuestra práctica:

- DOM API
- Fetch API
- Web Storage API
- Timers (setTimeout, setInterval)
- Geolocation
- Notifications
- Web Speech

Estas APIs están implementadas por el navegador, no por JS.

Ejemplo práctico

```
setTimeout(() => {
  console.log("Hola");
}, 1000);
```

¿Qué ocurre realmente?

1. El motor JS ve `setTimeout`
2. Lo pasa a la Web API de timers
3. El temporizador empieza fuera del motor JS
4. JS sigue ejecutando lo siguiente

JS no espera 1 segundo. JS sigue ejecutando.
Delegar significa: *"esto no lo gestiono yo"*.

Cuando termina → el callback va a la cola

Cuando el temporizador termina: La Web API NO ejecuta directamente el callback.

Lo coloca en una estructura llamada: **Task Queue (cola de tareas)**

En este momento:

- El callback está listo
- Pero todavía no se ejecuta

El Event Loop espera que la pila esté vacía

El Event Loop está constantemente preguntando:

```
¿La Call Stack está vacía?
```

Si la pila sigue ejecutando código síncrono,
el callback debe esperar.

Event Loop

Es el coordinador.
Su función:

- Vigilar la Call Stack
- Revisar la cola de tareas (callback queue)
- Ejecutar callbacks cuando la pila esté libre

Funcionamiento simplificado

1. JS ejecuta código síncrono
2. Cuando encuentra algo asíncrono (fetch, timer, evento click)
   → lo delega a Web API
3. Cuando termina
   → el callback va a la cola
4. Event Loop espera a que la pila esté vacía
5. Inserta el callback en la pila

Por este motivo:

- fetch() no bloquea
- la UI no se congela (si el código está bien hecho)
- existen promesas y async/await

UI Thread (render thread)

Aquí es donde se:

- Calcula layout
- Se aplican estilos
- Se pintan píxeles
- Se actualiza la pantalla

JavaScript comparte hilo con el renderizado. Si la interfaz se congela es porque se bloquea el hilo principal.

Ejemplo aplicado a nuestra práctica

Caso real: WebApp cultural de Baleares.
Usuario:

- Filtra por isla
- Se llama a fetch() para cargar JSON
- Se actualiza el DOM
- Se renderiza mapa interactivo

Lo que ocurre internamente:

1. Click → evento Web API
2. Event Loop ejecuta handler
3. Handler llama a fetch
4. Fetch delega a Web API de red

5. Respuesta → callback
6. JS actualiza DOM
7. UI thread recalcula layout
8. Pantalla se actualiza

Todo coordinado.
JS es el director de orquesta.

Single-thread: implicación directa

- JS en el browser es *mayoritariamente* single-thread (con matices)
- Si bloqueas el hilo, bloqueas la UI:
  - scroll se congela
  - clicks no responden
  - animaciones saltan

Consecuencia de ingeniería: tareas pesadas deben dividirse o evitarse.

Call stack (pila de ejecución)

- Cada llamada a función se apila
- Si hay recursión profunda o loops intensivos, bloqueas
- Errores típicos: "Maximum call stack size exceeded"

Event loop: el corazón del browser JS

Modelo:

- JS ejecuta código → termina → el event loop toma "tareas" pendientes
- Dos colas importantes:
  - **Task queue** (timers, eventos UI)
  - **Microtask queue** (promesas)

Regla práctica: microtasks suelen ejecutarse antes de volver a pintar.

Tipos en JS

Primitivos:

- `string`, `number`, `boolean`, `null`, `undefined`, `bigint`, `symbol`

Referencias:

- `object` (incluye arrays, funciones, fechas)

Arrays: operaciones de ingeniería

- `map` (transformación)

- `filter` (subconjunto)
- `reduce` (agregación)
- `find` (búsqueda)
- `sort` (ordenación, cuidado: muta)

Funciones: first-class + callbacks

- Una función se puede pasar como parámetro
- Esto habilita: eventos, callbacks, programación modular

Ejemplo conceptual:

- `onClick(handler)`
- `items.filter(predicate)`

Scope y closures (lo mínimo para no romperlo)

- `let/const` tienen scope de bloque
- closure: una función "recuerda" el scope donde fue creada

Ejemplo típico:

- handlers que usan variables externas
- fábricas de funciones para render

Módulos ES (arquitectura mínima)

- Separar código por responsabilidad
- `export` / `import`

Ejemplo:

- `data.js` (manejo de dataset)
- `ui.js` (render)
- `events.js` (handlers)
- `app.js` (bootstrap)

Evaluación: modularidad y estructura se valoran en el código

Estructura recomendada del proyecto (JS)

```
/js
  app.js        // inicialización
  state.js      // estado de la app
  ui.js         // render y templates
  events.js     // listeners y handlers
  data.js       // carga y consulta de datos
```

Objetivo: reducir acoplamiento, facilitar test y mantenimiento.

Patrón “Bootstrapping” (arranque)

Qué hace `app.js`:

1. esperar DOM listo
2. cargar datos iniciales (más adelante en T7)
3. inicializar estado
4. render inicial
5. registrar eventos

Ingeniería: separar “arranque” de “lógica”.

Eventos: modelo de interacción

- Un evento representa un suceso: click, input, submit, keydown
- Un handler es una función que reacciona
- Flujo típico:
  - evento → handler → actualizar estado → render

Regla: UI es función del estado (aunque sea sin framework).

Event propagation: capture → target → bubble

- Eventos se propagan por el DOM
- Esto habilita **event delegation** (clave para listas dinámicas)

Event delegation (concepto clave)

- Listener en un contenedor
- Detectas qué elemento interno disparó el evento

Ventajas:

- funciona con elementos creados dinámicamente
- escala mejor

DOM

DOM = representación en memoria del HTML, en forma de árbol de nodos.
Operaciones típicas:

- seleccionar (`querySelector`)
- leer atributos/texto
- crear nodos
- insertar/actualizar/eliminar

DOM (Document Object Model) es una representación en memoria del documento HTML como una estructura de árbol de nodos manipulable desde JavaScript.

No es HTML.
No es la página.
Es un modelo programable.

HTML vs DOM

HTML:

```
<ul>
  <li>Uno</li>
  <li>Dos</li>
</ul>
```

Cuando el navegador lo parsea:

1. El **parser** convierte el texto en nodos.
2. Construye un árbol.
3. Ese árbol es el DOM.

El navegador mantiene el DOM en memoria.

Estructura interna del DOM

Es un árbol jerárquico:

```
Document
└── html
    ├── head
    └── body
        └── ul
            ├── li
            └── li
```

Cada elemento es un nodo.
Tipos de nodos importantes:

- Document
- Element
- Text
- Comment
- Attribute

DOM es un grafo dirigido acíclico (DAG) con relaciones padre-hijo.

El DOM es una API

El DOM no es solo estructura.
Es una API expuesta por el navegador.
Ejemplos:

```
document.getElementById()
document.querySelector()
element.appendChild()
element.remove()
```

Estas funciones son parte de la DOM API.

DOM y Runtime JS

Relación arquitectónica:

- El motor JS ejecuta código.
- Las Web APIs incluyen la DOM API.
- El DOM vive fuera del motor JS.
- JS solicita modificaciones.
- El navegador recalcula layout y renderiza.

Flujo:
JS → DOM API → Actualización → Render
JS no dibuja píxeles -> Modifica el modelo.

Buenas prácticas de ingeniería DOM

Agrupar escrituras

Mal:

```
element style.width = "200px";
element.style.height = "300px";.
```

Mejor:

```
element.style.cssText = "width:200px;height:300px;";
```

Usar DocumentFragment

```
const fragment = document.createDocumentFragment();

for (...) {
  fragment.appendChild(crearNodo());
}

lista.appendChild(fragment);
```

Solo un reflow final.

Minimizar queries repetidas

Mal:

```
for (...) {
  document.querySelector(".item")
}
```

Mejor:

```
const item = document.querySelector(".item");
```

DOM y Event Loop

Cada manipulación DOM:

- Puede bloquear el hilo principal.
- Comparte hilo con render.
- Si haces cálculos pesados + DOM → UI se congela.

El DOM es single-threaded en el navegador clásico.

DOM y memoria

Cada nodo:

- Ocupa memoria.
- Tiene referencias.
- Puede generar leaks si no se eliminan listeners.

Ejemplo clásico:

```
element.remove();
```

Si no limpias referencias externas → no se libera memoria.

DOM como estado de aplicación

En aplicaciones pequeñas:
DOM = estado.

En aplicaciones grandes:
Se separa:

- Estado (JS)
- Representación (DOM)

De ahí surgen frameworks como:

- React
- Vue.js
- Angular

Porque manipular el DOM directamente a gran escala es costoso.

DocumentFragment y optimización de inserciones

Inserciones repetidas en el DOM real son costosas.
Ejemplo incorrecto:

```
for (let i = 0; i < 1000; i++) {
```

```
  lista.appendChild(crearNodo());
}
```

¿Qué ocurre internamente?

Cada `appendChild`:

1. Modifica el DOM
2. Puede invalidar el Render Tree
3. Puede provocar reflow
4. Puede provocar paint

Resultado:

- Múltiples recalculaciones
- Peor rendimiento
- Posibles frames perdidos

¿Qué es DocumentFragment?

```
const fragment = document.createDocumentFragment();
```

Un `DocumentFragment` es:
Un contenedor DOM ligero que vive en memoria pero NO está en el árbol visible.
Características:

- No pertenece al DOM principal
- No dispara layout
- No dispara paint
- Es temporal

Patrón correcto

```
const fragment = document.createDocumentFragment();

for (let i = 0; i < 1000; i++) {
  fragment.appendChild(crearNodo());
}

lista.appendChild(fragment);
```

¿Qué ocurre internamente?

Se crean nodos en memoria
Se insertan dentro del fragment (sin impacto visual)
Solo cuando se hace:

```
lista.appendChild(fragment);
```

Se insertan todos de golpe en el DOM real.
Resultado:
Un solo reflow potencial
Un solo repaint

Alternativa equivalente

También se puede hacer:

```
lista.innerHTML = htmlString;
```

Es rápido porque:

- Reemplaza todo de golpe
- Un solo parseo
- Un solo reflow

Cuándo usarlo

Ideal para:

- Listas grandes
- Resultados de búsqueda
- Carga de datos vía fetch
- Render inicial masivo

Ejemplo: creación de una card (DOM API)

```
function createCard(item) {
  const article = document.createElement("article");
  article.className = "card";
  article.dataset.id = item.id;

  const h2 = document.createElement("h2");
  h2.textContent = item.title;

  const p = document.createElement("p");
  p.textContent = item.summary;

  const btn = document.createElement("button");
  btn.type = "button";
  btn.textContent = "Ver detalle";

  article.append(h2, p, btn);
  return article;
}
```

textContent vs innerHTML

- `innerHTML` interpreta HTML → riesgo si el contenido viene de datos externos
- `textContent` inserta texto literal

Performance DOM: regla práctica

- Reducir escrituras al DOM
- Preferir construir y luego insertar (batching)
- Evitar leer layout en bucles (reflow)

Primer patrón útil:

```
const frag = document.createDocumentFragment();
items.forEach(i => frag.append(createCard(i)));
container.replaceChildren(frag);
```